

République du Sénégal



MINISTÈRE DE L'ENSEIGNEMENT SUPÉRIEUR ET DE LA RECHERCHE

DIRECTION GÉNÉRALE DE L'ENSEIGNEMENT SUPÉRIEUR



Présentation — Git & GitHub

Présenté par : **Ibrahima Khalilou Thiam**

Master 2 Génie Logiciel UDB

Matricule 003

À **M. Massamba LO**

Année académique 2025

Sommaire

1. [Introduction](#)
2. [Qu'est-ce que Git ?](#)
3. [Qu'est-ce que GitHub ?](#)
4. [Rôle et avantages](#)
5. [Installation de Git](#)
6. [Premiers pas avec Git \(commandes de base\)](#)
7. [Mise en place d'un projet — Scénario complet](#)
8. [Bonnes pratiques](#)
9. [Exemple de workflow collaboratif](#)
10. [Conclusion](#)

1. Introduction

Git (le système de contrôle de version) et GitHub (la plateforme d'hébergement et de collaboration). L'objectif est d'amener le lecteur de zéro à un workflow collaboratif complet, en expliquant chaque commande et chaque étape.

Pourquoi apprendre Git ?

Git permet de garder un historique fiable des modifications, d'expérimenter sans risquer de perdre le travail (grâce aux branches), et de collaborer efficacement. GitHub facilite la revue de code, l'intégration continue (CI), et le partage public/privé.

2. Qu'est-ce que Git ?

Git est un **système de contrôle de version distribué**. Contrairement à certains systèmes centralisés, chaque développeur possède une copie complète du dépôt (history + contenu) sur sa machine.

Concepts clés

- **Commit** : photographie de l'état du projet à un instant T.
- **Branch (branche)** : ligne de développement isolée.
- **Merge (fusion)** : intégration d'une branche dans une autre.
- **Remote (dépôt distant)** : version en ligne du dépôt (ex. : GitHub).

Le cycle de vie simplifié :

Modifier des fichiers → `git add` pour indexer → `git commit` pour enregistrer → `git push` pour envoyer au remote.



3. Qu'est-ce que GitHub ?

GitHub est une plateforme qui héberge des dépôts Git et y ajoute des fonctionnalités collaboratives : issues, pull requests, actions CI, pages pour hébergement statique, etc.

Fonctionnalités importantes

- **Pull Requests** : mécanisme de revue et de demande de fusion.
- **Issues** : suivi des tâches et bugs.
- **Actions** : workflows d'intégration/déploiement continus.
- **Pages** : hébergement de sites statiques (ex. : documentation).

Différence pratique :

Git s'exécute sur votre machine ; GitHub est un service web où vous publiez vos dépôts pour collaborer et publier.



4. Rôle et avantages de Git & GitHub

a) Collaboration

Les branches et les pull requests permettent d'isoler le travail, de faire des revues et de discuter des changements avant de fusionner.

b) Sécurité et sauvegarde

Chaque commit est identifié par un hash (SHA-1). Les données sont difficiles à corrompre sans être détectées.

c) Productivité

Automatisation (CI/CD), intégrations IDE, outils de gestion de projet.

d) Communauté

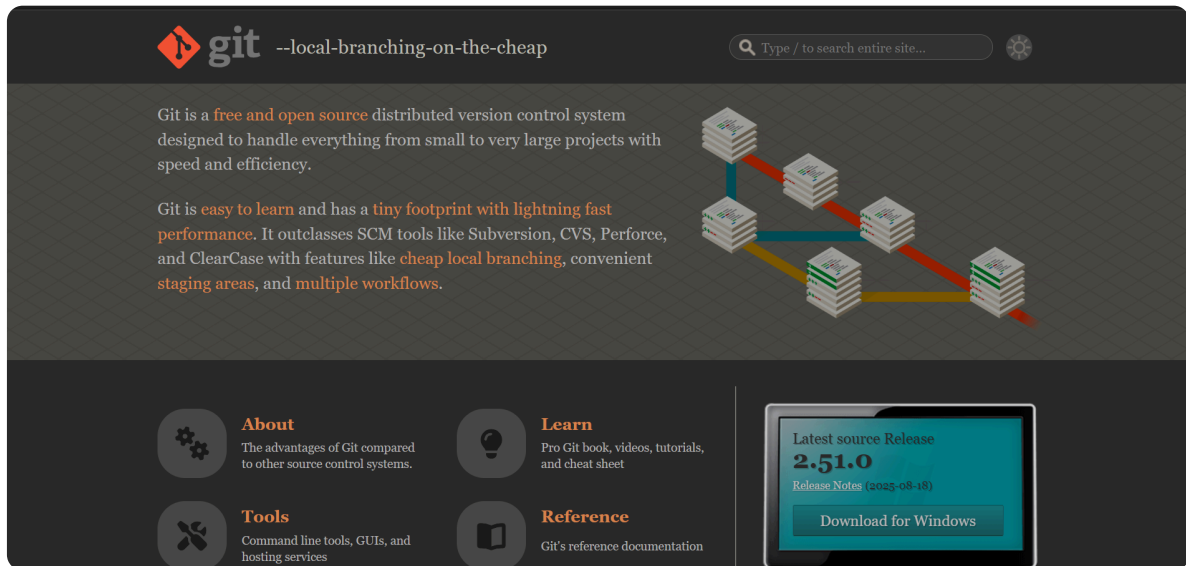
Large écosystème de projets open-source, templates et intégrations.

 git	 GitHub
1. It is a software	1. It is a service
2. It is installed locally on the system	2. It is hosted on Web
3. It is a command line tool	3. It provides a graphical interface
4. It is a tool to manage different versions of edits, made to files in a git repository	4. It is a space to upload a copy of the Git repository
5. It provides functionalities like Version Control System Source Code Management	5. It provides functionalities of Git like VCS, Source Code Management as well as adding few of its own features

5. Installation de Git (Windows, macOS, Linux)

Windows

- Télécharger depuis git-scm.com
- Lancer l'installateur, choisir l'éditeur préféré (VS Code conseillé) et activer Git from CLI
- Vérifier : `git --version`



macOS

Option 1 : Homebrew

```
/bin/bash -c  
"$(curl -fsSL https://raw.githubusercontent.com/Homebrew/install/HEAD/install.sh)"  
brew install git
```

Option 2 : Installateur officiel depuis git-scm.com.

Linux

Ubuntu / Debian

```
sudo apt update  
sudo apt install git
```

Fedora / CentOS

```
sudo dnf install git
```

Arch Linux / Manjaro

```
sudo pacman -S git
```

```
Session Actions Éditer Vue Aide
(creator@creator)-[~]
$ sudo apt install git
Atteint : 1 http://http.kali.org/kali kali-rolling InRelease
713 paquets peuvent être mis à jour. Exécutez « apt list --upgradable » pour les voir.

(creator@creator)-[~]
$ sudo apt install git
Les paquets suivants ont été installés automatiquement et ne sont plus nécessaires :
binutils-mingw-w64-i686      libaio1t64      python3-qasync
binutils-mingw-w64-x86_64    libapache2-mod-php  python3-qrcode
dnsmasq                     liblua5.1-2      python3-serial-asyncio
dsniff                      liblua5.1-common  python3-smmap
ettercap-common             libnids1.21t64    python3-tld
ettercap-graphical          libsmb2-6         python3-yaswfp
figlet                      medusa           rsh-redone-client
finger                     mingw-w64-common  smtp-user-enum
gcc-mingw-w64-base          mingw-w64-i686-dev  sparte-scripts
gcc-mingw-w64-1686-win32    mingw-w64-x86_64-dev  toilet-fonts
gcc-mingw-w64-1686-win32-runtime  oracle-instantclient-basic  unicornscan
gcc-mingw-w64-x86_64-win32  python3-gitdb     urlscan
gcc-mingw-w64-x86_64-win32-runtime  python3-pefile    wapiti
imagemagick-7.q16          python3-pyfiglet   python3-pyshodan

Veuillez utiliser « sudo apt autoremove » pour les supprimer.

Installation de :
git

Paquets suggérés :
git-doc git-email git-gui gitk gitweb git-cvs git-svn

Sommaire :
Mise à niveau de : 0. Installation de : 1Supprimé : 0. Non mis à jour : 713
Taille du téléchargement : 9 259 kB
Espace nécessaire : 51,4 MB / 8 995 MB disponible

Réception de : 1 http://mirror.es.cdn-perfprod.com/kali kali-rolling/main amd64 git amd64 1:2.51.0-1 [9 2
59 kB]
9 259 ko réceptionnés en 3s (2 732 ko/s)
Sélection du paquet git précédemment désélectionné.
(Lecture de la base de données... 384406 fichiers et répertoires déjà installés.)
```

Vérifier : **git --version**

```
Session Actions Éditer Vue Aide
(creator@creator)-[~]
$ git --version
git version 2.51.0

(creator@creator)-[~]
$
```

Peu importe l'OS, les commandes Git suivantes sont identiques. Seules diffèrent les étapes d'installation.



Mac OS X



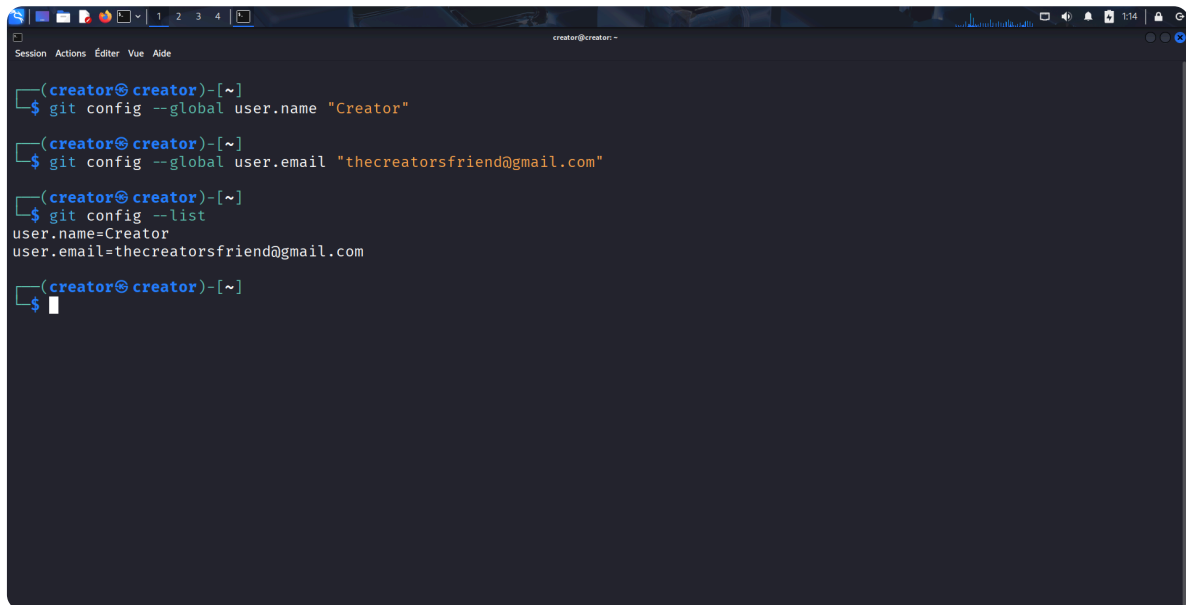
Windows



Linux/Unix

6. Premiers pas avec Git — commandes de base

6.1 Configuration initiale

A screenshot of a terminal window with a dark background. The prompt is '(creator@creator)-[~]'. The user enters three commands: 'git config --global user.name "Creator"', 'git config --global user.email "thecreatorsfriend@gmail.com"', and 'git config --list'. The output of the last command shows 'user.name=Creator' and 'user.email=thecreatorsfriend@gmail.com'. The prompt returns to '(creator@creator)-[~]'.

```
# Configurer votre nom (affiché dans les commits)
git config --global user.name "Votre Nom"

# Configurer votre e-mail (affiché dans les commits)
git config --global user.email "vous@exemple.com"

# Optionnel : afficher la configuration courante
git config --list
```

Explication :

Ces paramètres renseignent l'auteur des commits. L'option `--global` applique la configuration pour tout l'utilisateur sur la machine. Sans ces paramètres, Git vous demandera de les définir avant de committer.

Note : Pour un projet professionnel, on peut configurer des valeurs locales (sans `--global`) dans le dépôt seulement : `git config user.email "autre@exemple.com"`.

6.2 Créer un dépôt local

```
Session Actions Éditer Vue Aide
creator@creator: ~/mon-projet

$ mkdir mon-projet

$ cd mon-projet

$ git init
astuce : Utilisation de 'master' comme nom de la branche initiale. Le nom de la branche
astuce : par défaut peut changer. Pour configurer le nom de la branche initiale
astuce : pour tous les nouveaux dépôts, et supprimer cet avertissement, lancez :
astuce :
astuce :     git config --global init.defaultBranch <nom>
astuce :
astuce : Les noms les plus utilisés à la place de 'master' sont 'main', 'trunk' et
astuce : 'development'. La branche nouvellement créée peut être renommée avec :
astuce :
astuce :     git branch -m <nom>
astuce :
astuce : Disable this message with "git config set advice.defaultBranchName false"
Dépôt Git vide initialisé dans /home/creator/mon-projet/.git/

$ ls -a
.  ..  .git

$
```

Créer un dossier et initialiser Git

```
mkdir mon-projet
```

```
cd mon-projet
```

```
git init
```

Résultat : création d'un dossier .git qui contient l'historique local

Explication :

`git init` crée un dépôt Git dans le dossier courant. Cela n'envoie aucune donnée en ligne — vous avez un dépôt local prêt à recevoir des commits.

6.3 Ajouter des fichiers et faire un commit

```
Session Actions Éditer Vue Aide
creator@creator: ~/mon-projet

$ nano index.html

$ nano style.css

$ git add index.html style.css

$ git commit -m "Initialisation du projet : ajout des fichiers index et style"
[master (commit racine) 831a7ac] Initialisation du projet : ajout des fichiers index et style
2 files changed, 14 insertions(+)
create mode 100644 index.html
create mode 100644 style.css

$ git status
Sur la branche master
rien à valider, la copie de travail est propre

$ git log --oneline --graph --decorate
* 831a7ac (HEAD -> master) Initialisation du projet : ajout des fichiers index et style

$
```

```
# Ajouter des fichiers pour suivi (staging)
git add index.html style.css

# Enregistrer ces changements dans l'historique (commit)
git commit -m "Initialisation du projet : ajout des fichiers index et style"

# Vérifier l'état du dépôt
git status

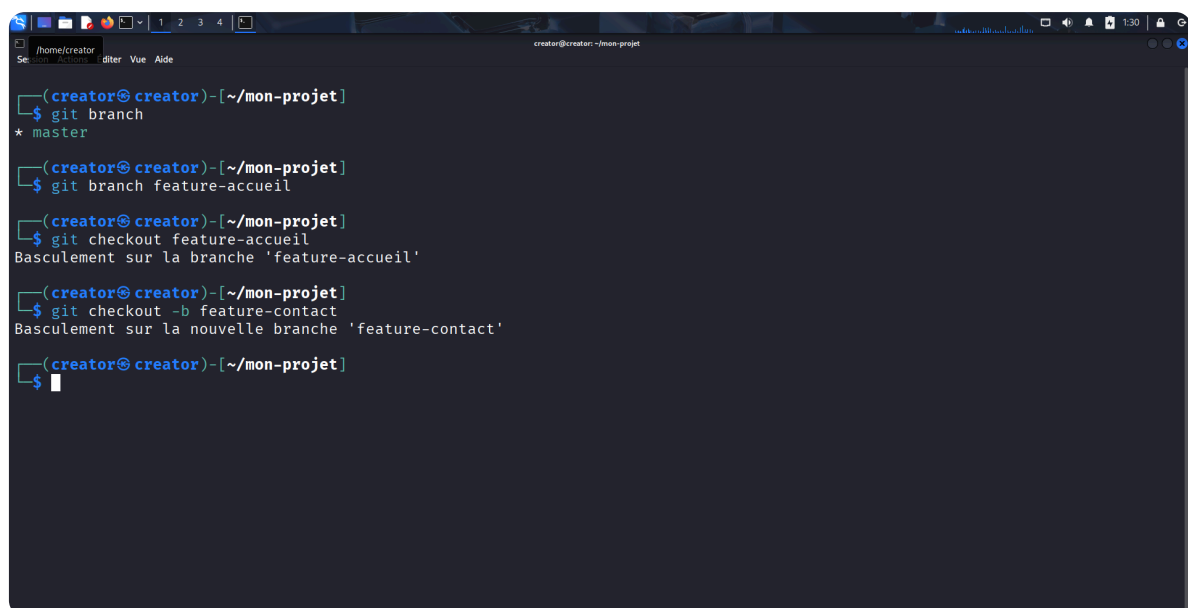
# Voir l'historique des commits
git log --oneline --graph --decorate
```

Explication :

- **git add** place des modifications dans l'index (staging area).
- **git commit** enregistre les modifications indexées dans l'historique local comme un commit.
- **git status** montre les fichiers modifiés, indexés ou non.
- **git log** permet de consulter l'historique ; les options **--oneline** et **--graph** rendent l'affichage plus lisible.

Erreur courante : Faire **git commit** sans **git add** n'enregistrera pas vos dernières modifications. Vérifiez toujours **git status**.

6.4 Branches — créer et basculer



```
creator@creator: ~/mon-projet
$ git branch
* master

creator@creator: ~/mon-projet
$ git branch feature-accueil

creator@creator: ~/mon-projet
$ git checkout feature-accueil
Basculement sur la branche 'feature-accueil'

creator@creator: ~/mon-projet
$ git checkout -b feature-contact
Basculement sur la nouvelle branche 'feature-contact'

creator@creator: ~/mon-projet
$
```

```
# Lister les branches locales
git branch

# Créer une nouvelle branche
git branch feature-accueil

# Basculer sur la branche
git checkout feature-accueil

# Alternative (crée et bascule en un seul pas)
git checkout -b feature-contact
```

Pourquoi les branches ?

Les branches permettent d'isoler une nouvelle fonctionnalité ou une correction de bug. La branche **main** (ou **master**) reste stable pendant que vous travaillez sur une branche secondaire.

6.5 Fusionner une branche

```
# Se placer sur la branche de destination (ex: main)
git checkout main

# Fusionner la branche feature-accueil dans main
git merge feature-accueil

# Résoudre conflits si nécessaire
# Après résolution : git add
# Puis : git commit (pour terminer la fusion si non automatique)
```

Conflits :

Si deux personnes ont modifié la même portion de fichier, Git signale un conflit. Il faut ouvrir le fichier, choisir la version souhaitée, puis ajouter et committer la résolution.

6.6 Lier un dépôt distant et pousser

```
# Ajouter un remote nommé 'origin' (ex. GitHub)
git remote add origin https://github.com/utilisateur/mon-projet.git

# Renommer la branche par défaut (optionnel)
git branch -M main

# Envoyer la branche 'main' vers origin
git push -u origin main

# Pour pousser une branche nommée feature-xyz
git push origin feature-xyz
```

Explication :

`git remote add origin ...` lie le dépôt local à un dépôt distant. `git push -u origin main` envoie la branche et définit l'upstream pour faciliter les futurs pushes/pulls.

7. Mise en place d'un projet Git & GitHub — scénario complet expliqué

Ce scénario illustre toutes les étapes depuis la création locale jusqu'à la mise en ligne sur GitHub et la création d'une fonctionnalité via branche + pull request.

Étape 1 — Création locale

```
# Créer un répertoire de projet et initialiser
mkdir projetGit
cd projetGit
git init

# Créer un fichier simple
echo "<h1>Mon premier projet Git</h1>" > index.html

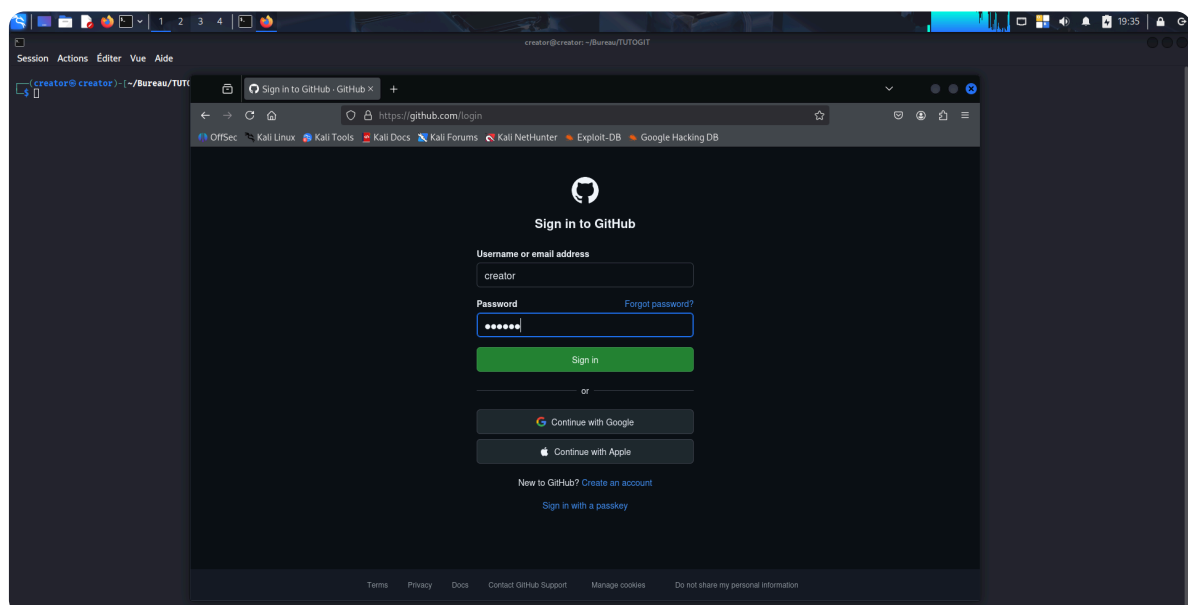
# Suivre et committer
git add index.html
git commit -m "Création du fichier index.html"
```

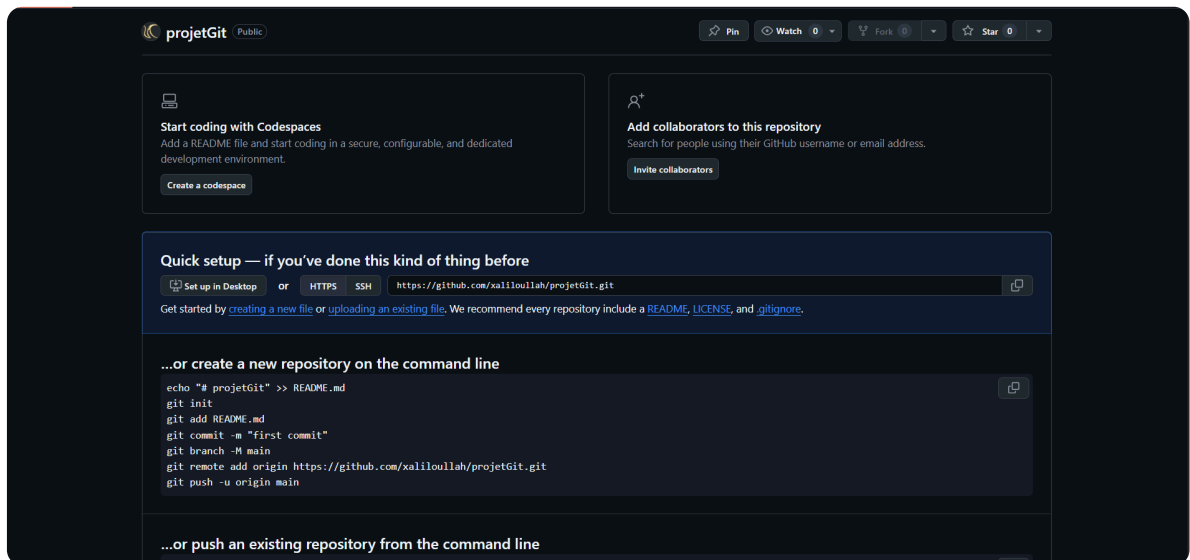
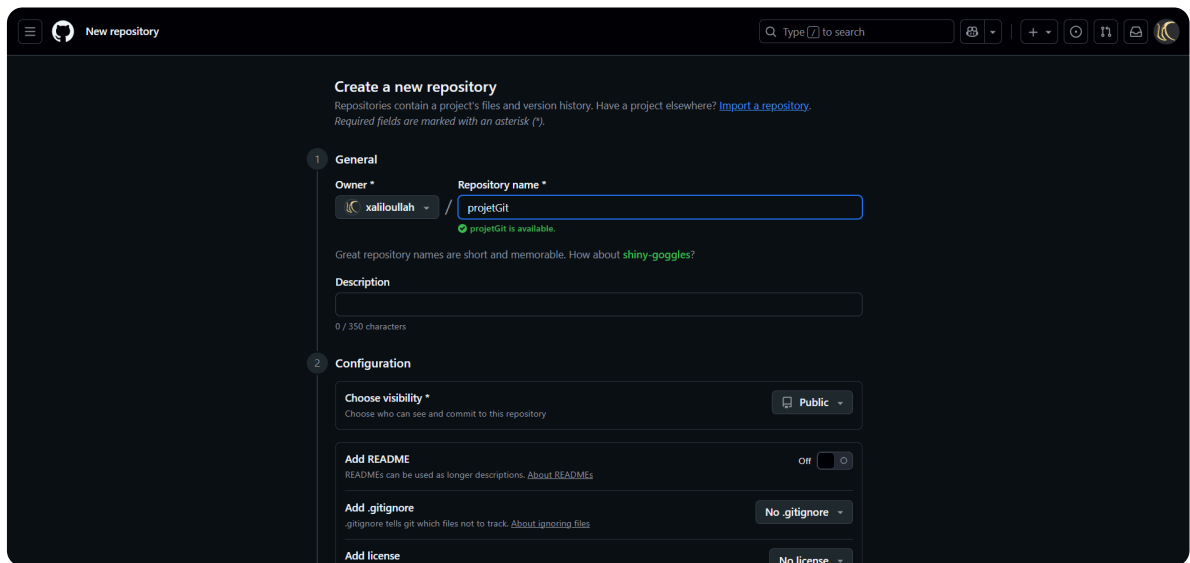
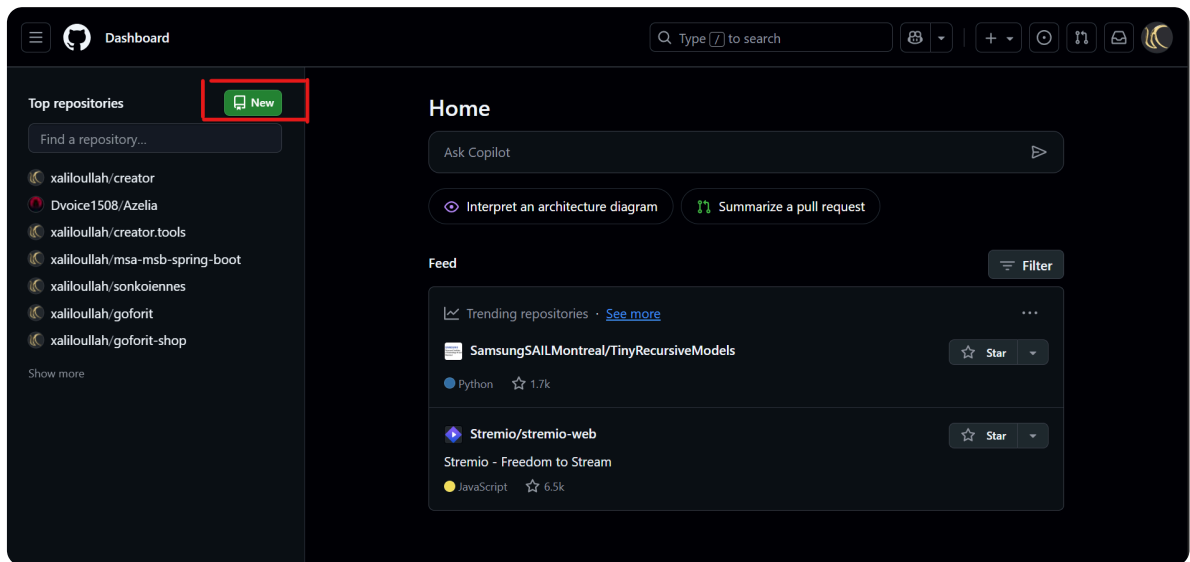
Explication :

Ce commit initialise le projet. À partir de maintenant, toute modification de `index.html` pourra être tracée dans l'historique.

Étape 2 — Création du dépôt GitHub

Sur **github.com** : cliquer sur *New Repository*, nommer le dépôt `projetGit`, choisir public/privé, puis créer.





Étape 3 — Lier local → distant

```
# Ajouter le remote (url HTTPS)
git remote add origin https://github.com/ton-username/projetGit.git

# Renommer branche et pousser la première fois
git branch -M main
git push -u origin main
```

Note :

Tu peux également utiliser SSH (git@github.com:username/projetGit.git) si tu as configuré une clé SSH pour GitHub.

Étape 4 — Développer une nouvelle fonctionnalité

```
# Créer une branche de fonctionnalité
git checkout -b feature-contact

# Ajouter une nouvelle page
echo "<p>Contactez-nous</p>" > contact.html

# Ajouter et committer
git add contact.html
git commit -m "Ajout de la page contact"

# Pousser la branche sur GitHub
git push origin feature-contact
```

Explication :

La branche **feature-contact** est désormais en ligne et prête à être examinée via une Pull Request sur GitHub.

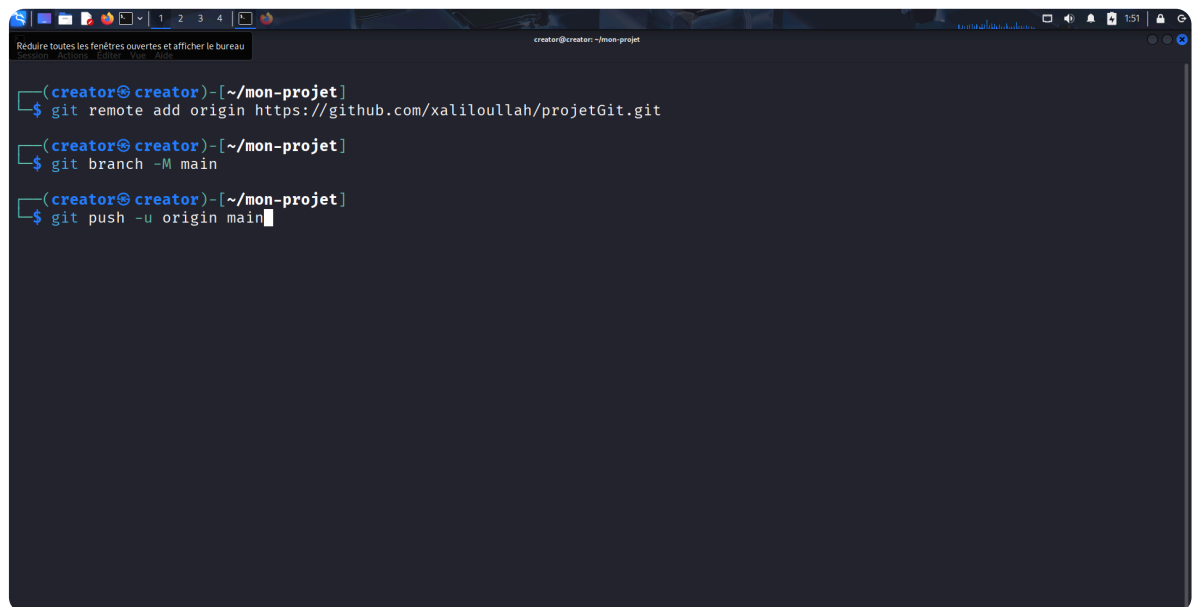
Étape 5 — Créer une Pull Request (PR)

Sur l'interface GitHub : comparer la branche **feature-contact** avec **main**, ouvrir une PR, ajouter une description, assigner des réviseurs et lancer la revue.

Étape 6 — Fusionner la PR

Après approbation, fusionner via l'interface (merge). Ensuite récupérer les changements localement :

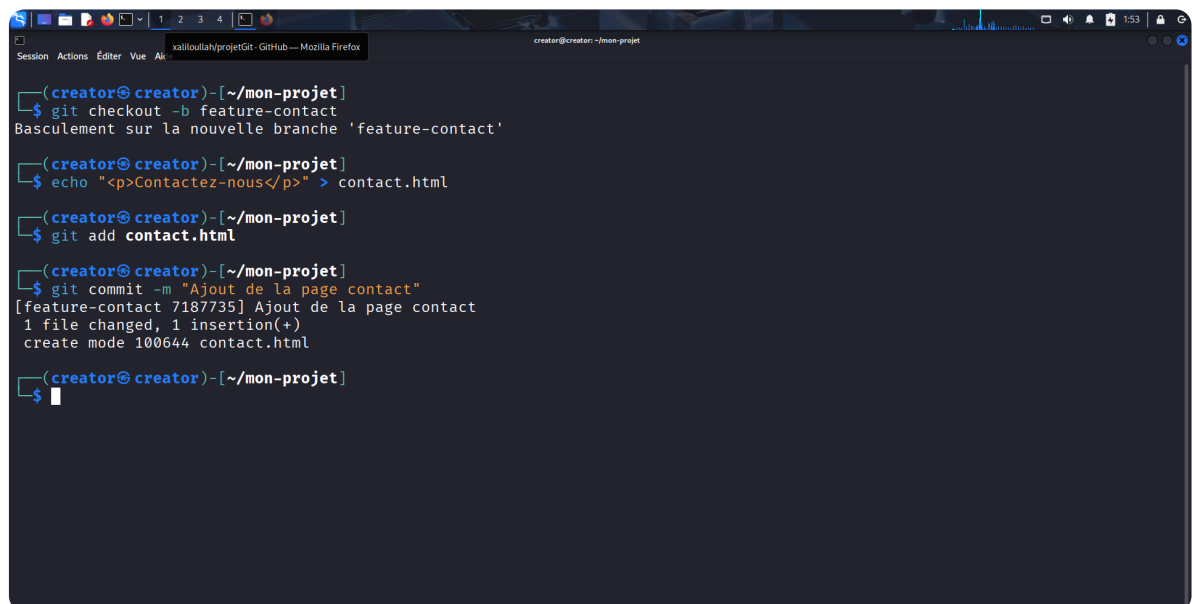
```
# Retourner sur main et récupérer les modifications depuis origin
git checkout main
git pull origin main
```



```
(creator@creator)-[~/mon-projet]
$ git remote add origin https://github.com/xaliloullah/projetGit.git

(creator@creator)-[~/mon-projet]
$ git branch -M main

(creator@creator)-[~/mon-projet]
$ git push -u origin main
```



```
(creator@creator)-[~/mon-projet]
$ git checkout -b feature-contact
Basculement sur la nouvelle branche 'feature-contact'

(creator@creator)-[~/mon-projet]
$ echo "<p>Contactez-nous</p>" > contact.html

(creator@creator)-[~/mon-projet]
$ git add contact.html

(creator@creator)-[~/mon-projet]
$ git commit -m "Ajout de la page contact"
[feature-contact 7187735] Ajout de la page contact
1 file changed, 1 insertion(+)
create mode 100644 contact.html

(creator@creator)-[~/mon-projet]
$
```

8. Bonnes pratiques de travail collaboratif

Branches et conventions

Adoptez une convention claire : `feature/...`, `fix/...`, `hotfix/...`, etc. Ex : `feature/ajout-accueil`.

Messages de commit

Rédigez des messages concis mais descriptifs : première ligne courte (<50 char), puis description détaillée si nécessaire.

Fichiers sensibles

Utilisez `.gitignore` pour exclure les secrets, dossiers temporaires et dépendances volumineuses.

```
# Exemple .gitignore basique
node_modules/
.env
.DS_Store
*.log
/build/
dist/
```

Astuce sécurité : Si un secret est commit accidentellement, invalidez-le (par ex. régénérer la clé) — ne comptez pas sur un "git rm" seul pour résoudre le problème.

9. Exemple complet de workflow collaboratif (pas à pas)

Résumé du workflow

1. Cloner le dépôt : `git clone`
2. Créer une branche locale : `git checkout -b`
3. Faire des commits atomiques (un seul objectif par commit)
4. Pousser la branche : `git push origin ma-branche`
5. Ouvrir une PR sur GitHub pour revue
6. Corriger les remarques, mettre à jour la PR
7. Fusionner et supprimer la branche distante

```
# Exemple concret
git clone https://github.com/organisation/projet.git
cd projet
git checkout -b feature/amelioration-style
# Travailler localement
git add .
git commit -m "feature: amélioration de la mise en page de la page d'accueil"
git push origin feature/amelioration-style
# Ouvrir PR -> Revue -> Merge
```

Pourquoi des commits atomiques ?

Ils facilitent la revue, la compréhension de l'historique et la possibilité de revenir en arrière sur une fonctionnalité précise.

10. Conclusion

Git & GitHub sont des outils fondamentaux pour tout développeur moderne. La maîtrise des commandes de base, la compréhension des branches et la pratique d'un workflow collaboratif permettent de travailler efficacement en équipe et de maintenir un historique fiable des projets.

Prochaines étapes conseillées :

- Pratiquer en créant un dépôt personnel et en simulant PRs.
- Apprendre les commandes avancées : `rebase`, `cherry-pick`, `bisect`.
- Découvrir GitHub Actions pour automatiser les tests et déploiements.

Ressources & Liens utiles

- [Site officiel Git](#)
- [Docs GitHub](#)
- [Pro Git \(livre, FR\)](#)

Récapitulatif commandes essentielles

- `git init` — initialiser
- `git clone` — cloner
- `git add` — indexer
- `git commit` — enregistrer
- `git push` / `git pull`
- `git branch` / `git checkout`

Annexes — Erreurs courantes et résolution

Erreur : "fatal: refusing to merge unrelated histories"

```
# Solution (si vous savez ce que vous faites)
git pull origin main --allow-unrelated-histories
```

Cette erreur survient lorsque l'historique local et distant n'ont pas d'ancêtre commun (ex. : dépôt distant initialisé avec README). Utiliser cette option avec prudence.

Erreur : "permission denied (publickey)"

```
# Vérifier la clé SSH locale et l'ajouter à GitHub
# Générer (si besoin)
ssh-keygen -t ed25519 -C "votre_email@example.com"
# Copier la clé publique et l'ajouter à GitHub (Settings > SSH and GPG keys)
cat ~/.ssh/id_ed25519.pub
```

Erreur : commit non voulu déjà poussé

```
# Revenir en arrière localement (si non poussé)
git reset --soft HEAD~1

# Si déjà poussé, utiliser revert plutôt que reset pour garder l'historique :
git revert commit_hash
```

Important : éviter les re-écritures d'historique (`git push --force`) sur des branches partagées sans coordination ; cela casse l'historique des autres contributeurs.